CS637 · RESEARCH PAPER EVALUATION

Evaluating eFlx

Energy flexibility provisioning for e-taxi fleets: re-implemented, reproduced, critiqued and extended

TEAM

Ghada Suleiman · 261330004
Rishikesh Sawant · 261110052
Suraj Singh Raman · 261110067
IIT Kanpur

PAPER

L. Zhou, Y. Zhao, Y. Yuan, C. Xu, S. Lin · ICCPS 2025

OUTLINE

First the paper, then our work

Part	Slides	Presented by
Introduction and outline	1–2	Ghada Suleiman
The paper: demand response and the problem	3–6	Ghada Suleiman
The paper: the model and the eFlx solution	7–9	Ghada Suleiman
Evaluating the paper: its claims and our critique	10–11	Suraj Singh Raman
Implementing the technical idea	12–16	Suraj Singh Raman
Reproducing the paper’s results	17–20	Suraj (17), Rishikesh (18–20)
New use cases and examples	21–22	Rishikesh Sawant
New ideas and extended implementation	23–24	Rishikesh Sawant
Conclusion and contributions	25–26	All three

FROM THE PAPER what the authors did **OUR WORK** what we did ourselves

1 · INTRODUCTION

A taxi fleet as a power plant on wheels

Demand response (DR)

When the grid is short of power (a generator fails, a heat wave), it pays large consumers to cut their demand for a while.

Why a taxi fleet is special

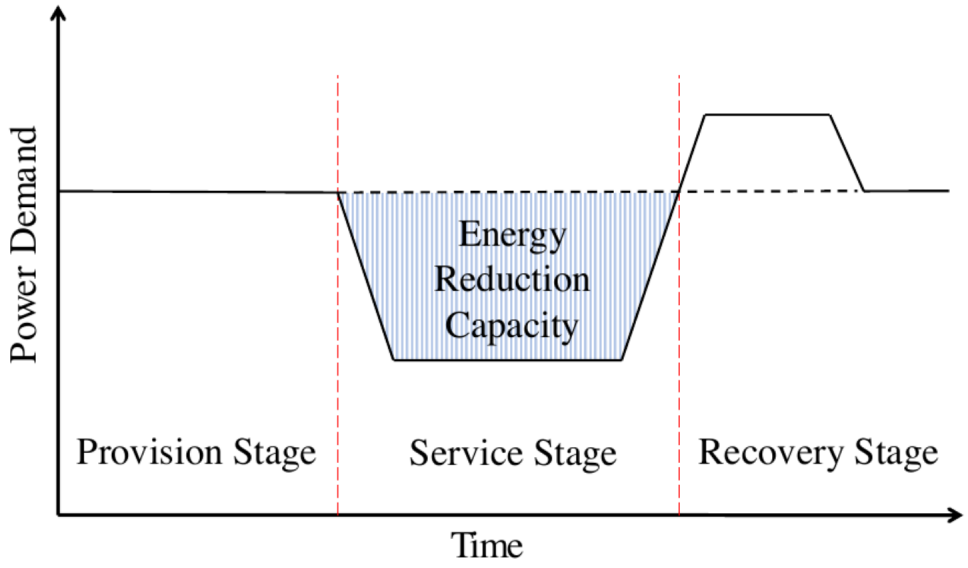
A factory must stop working to cut load. A fleet has batteries, and one coordinator can move **when and where** thousands of taxis charge, without passengers noticing.

50+ kWh	used by one e-taxi per day
~100 MW	charging load of a 13,000-taxi Manhattan fleet
10 min	to start cutting after an emergency call (NYISO DSASP)

FROM THE PAPER

1 · INTRODUCTION

A DR program has three stages



1 · Provision

The fleet has a DR contract but no call yet: keep serving, no extra charging, get ready

2 · Service (2 hours)

The call comes: cut charging, live on stored energy, still serve passengers

3 · Recovery

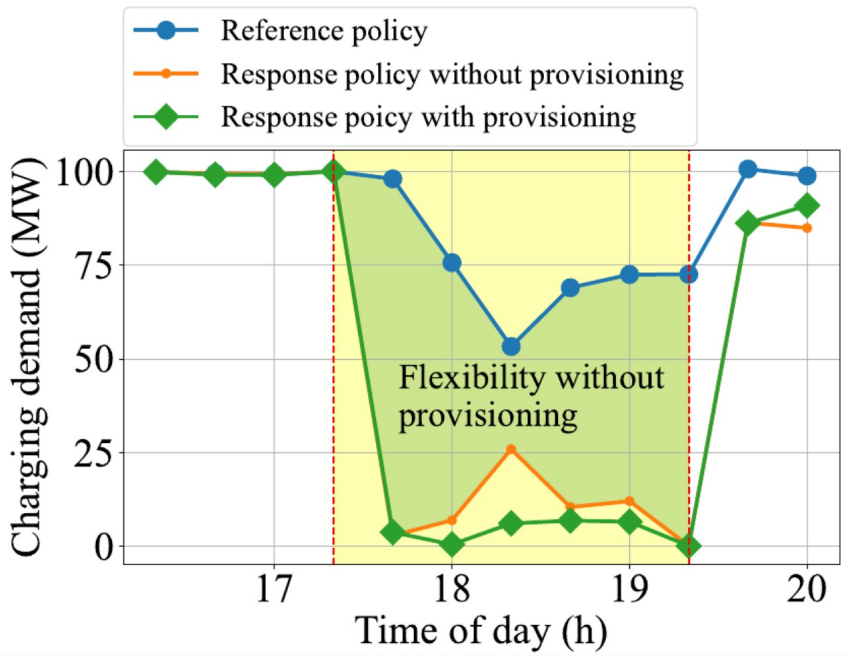
Recharge and return to normal

Key point: the call can come at any time, so the fleet must stay ready in every 20-minute slot.

FROM THE PAPER

1 · INTRODUCTION

Reacting helps; preparing helps more



127.86 MWh

cut in 2 hours by a fleet that only **reacts** when called

139.37 MWh

cut by a fleet that **prepared** in advance: +11.5 MWh, +9%

Same passengers served in all cases. The paper is about how to prepare as well as possible.

2 · PROBLEM STATEMENT

Be ready for a call that can come any time

THE QUESTION

How should the fleet dispatch and charge taxis **now**, before any call, so that whenever a call arrives it can cut as much charging as possible, without losing passengers?

FLEXIBILITY (EQ. 6)

charging if no call came – charging when responding

over the 2-hour window, in MWh. It comes from **spare taxis** and **stored battery energy**.

RULES

1 · Call time unknown
Be ready in every 20-min slot

2 · No extra charging first
Never above normal service (Eq. 8d)

3 · No worse service
Serve as many passengers as normal

Hard case studied: the evening peak, DR window 17:20–19:20, when passenger and grid demand are both highest.

3 · THE PAPER'S SOLUTION

The model: counting taxis, not tracking them

SPACE
City split into regions

TIME
20-minute slots

ENERGY
15 battery levels

Vacant
Cruising, can pick up or be sent elsewhere

Occupied
Carrying a passenger

Charging
+3 levels per slot at a station

Waiting
Parked at a station, using no energy

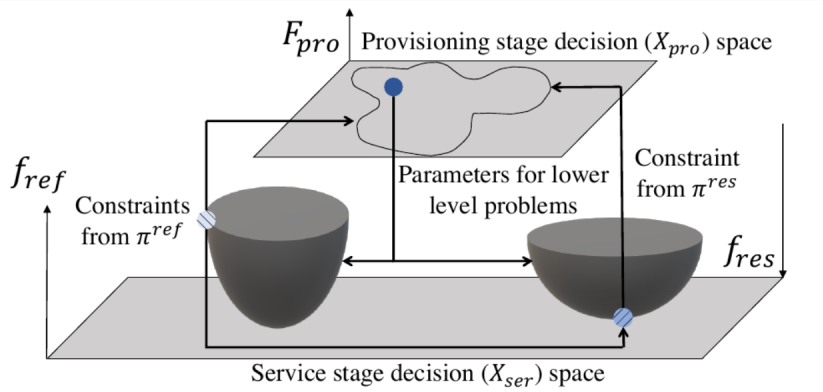
Decisions each slot: send vacant taxis to regions or chargers, and choose how many charge.

Dynamics (Eq. 1–4) are linear in these counts, so every plan is a linear program.

3 · THE PAPER'S SOLUTION

Why it is hard: the yardstick moves

Flexibility = baseline − response. But the baseline is "what the fleet would do with no call", which is the very policy being optimised. Its baseline needs a baseline, and so on.



(●, ▨, ▨): A feasible bi-level solution for upper-level optimization problem

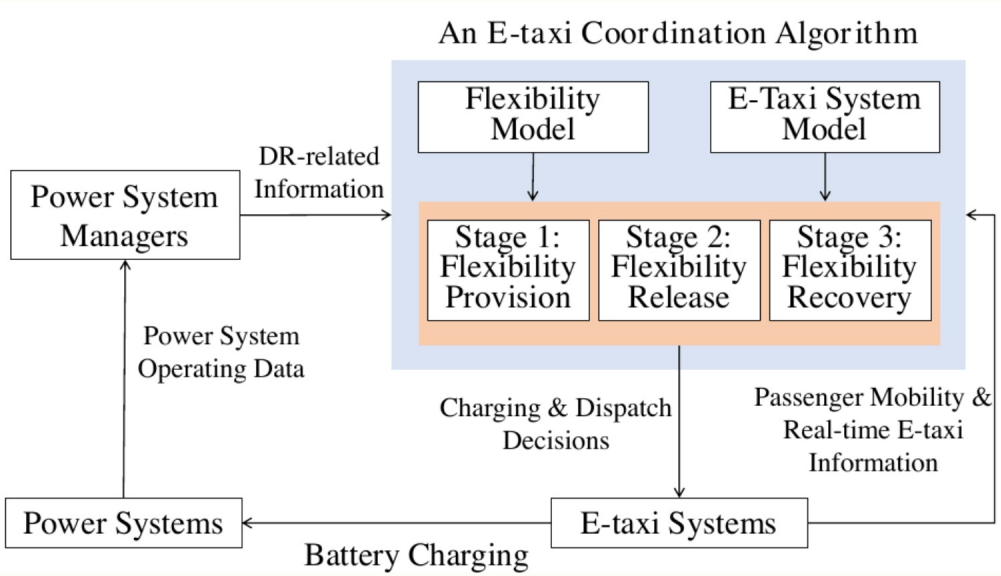
Problem 2: infinite-level
Baseline = the policy itself; cannot be solved

Problem 4: bi-level
Baseline = normal service; exact MILP has 1,482,570 variables

Problem 5: one LP
Predict normal service's numbers, so both levels merge

3 · THE PAPER'S SOLUTION

eFlx: a closed-loop fleet controller



Algorithm 1: provisioning

Solve the LP, re-predict normal service, solve again, until the plan settles (~3–4 rounds)

Algorithm 2: MPC loop

Every 20 min: read the fleet, provision, respond or serve normally, apply only this slot's orders

Always assumes

the call may come in the next slot ($\tau = 1$)

FROM THE PAPER

4 · EVALUATING THE PAPER

What the paper claims

−19.98%

charging during the DR window, vs TRC

+5.52%

flexibility vs TRC

0.52%

from an Oracle that knows when DR starts

~2%

extra idle driving and waiting; service unchanged

SETUP

~13,000 Manhattan e-taxis, real NYC trip data, DR window 17:20–19:20

BASELINES

TRC (normal), R2I (park idle taxis at stations), TES, Oracle

4 · EVALUATING THE PAPER

Our critical reading: strengths and gaps

Strengths

- Important, under-studied problem: fleets as grid helpers
- Elegant escape from the infinite-level loop
- Fast: one LP, real time on a PC
- Fair tests: same response policy, cap and service level

Gaps we found → what we did

Flexibility is measured against each policy's own baseline	shown, REPORT.md
Always assumes DR in the next slot	P-eFlx, slide 23
"Controlled service compromise" never measured	measured, slide 24
Only cuts load; grids also need extra charging	tested, slide 22
Missing parameters (β, η, battery, ports, days)	chosen, NOTES.md
Battery lasts ~4.7 h: 3–4× the paper's 50 kWh/day	noted
Fixed trip probabilities; the "Oracle" is not a bound	noted
Qv/Qo text swapped vs Eq. 2b–2c; free index in Eq. 6	noted

5 · IMPLEMENTATION

Rebuilding eFlx with no code released

Paper	Our code	In plain words
Eq. 1–4, 5e–5g	eflx/model.py	The city's rules: how taxis move
Problems 1, 3, 5 · Alg. 1 · baselines	eflx/policies.py	The strategies, eFlx included
Alg. 2 (MPC) · Eq. 6	eflx/sim.py	Plan, act, repeat; score flexibility
Same LPs at Manhattan scale	eflx/fastplan.py	Fast engine, ~20× faster
NYC trips → model inputs	scripts/get_nyc_data.py	Real demand and trip patterns

EQ. (1) AS CODE

```
# every available taxi gets exactly one decision
sum(x[l], axis=1) + sum(y[l], axis=1) == V[k][l]
```

STACK

Python · cvxpy · HiGHS LP solver · pandas · matplotlib, on a laptop

5 · IMPLEMENTATION

How we know our code is right

100 / 100

taxis conserved in every slot (checked by the simulator)

4 / 4

test states: readable and fast LPs reach identical optima

~20×

faster with the sparse builder

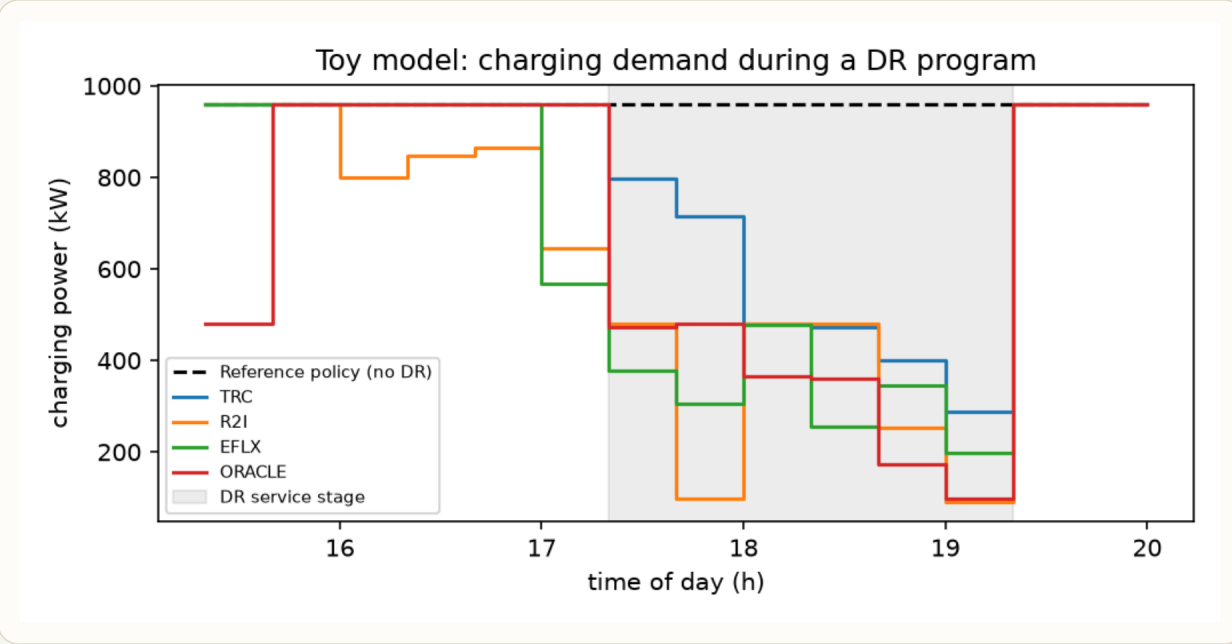
```
$ python tests/test_fastplan.py
case 0: reference obj 233.2720, response obj 0.0000 → identical
case 1: reference obj 258.8344, response obj 30.6614 → identical
case 2: reference obj 267.7137, response obj 49.2266 → identical
case 3: reference obj 258.5566, response obj 40.1378 → identical
FastPlan matches the cvxpy spec
```

5 · TOY MODEL

Toy model: the core idea reproduces

2 regions · 2 stations · 100 taxis · DR 17:20–19:20 · with charging limit

Policy	Flexibility	DR charging
TRC	870 kWh	1050 kWh
R2I	1256 kWh	628 kWh
eFlx	1267 kWh	653 kWh
Oracle	1270 kWh	650 kWh



eFlx: **−38%** DR charging vs TRC, at 100% service

5 · FINDING

Same optimum, different answers

Toy DR-window charging from our two LP builders: each reaches the same optimal objective at every step

Policy	Fast builder	cvxpy builder	Change
TRC	1050 kWh	1083 kWh	+3%
R2I	628 kWh	668 kWh	+6%
eFlx	653 kWh	712 kWh	+9%
Oracle	650 kWh	561 kWh	−14%

Why

The LPs have many tied optimal solutions; the solver picks one

So what

A “0.52% from Oracle” gap can sit inside solver noise

Also found

The Oracle must see the whole DR window, or it loses to eFlx

5 · DEMO

Demo: anyone can rerun it

STEP 1

Open the GitHub repo and its README

STEP 2 · ~4 s

Run the validation test

STEP 3 · ~7 s

Run the toy experiment, open its figure

```
$ git clone https://github.com/ascyrax/cs637-eflx && cd cs637-eflx
$ python tests/test_fastplan.py
FastPlan matches the cvxpy spec
$ python toy_demo.py
sanity OK: 100 taxis conserved over 15 slots
eflx flex 1266.9 kWh DR charge 653.1 kWh served 100.00%
plot saved to results/toy_fig8.png
```

6 · REPRODUCTION

Real data: NYC taxi trips, March 2024

2.01 M
weekday Manhattan trips over 21 days

12
regions, clustered from 62 taxi zones

~2,430
trips per 20-min slot at the evening peak

84%
of trips finish within one slot

FROM THE PAPER

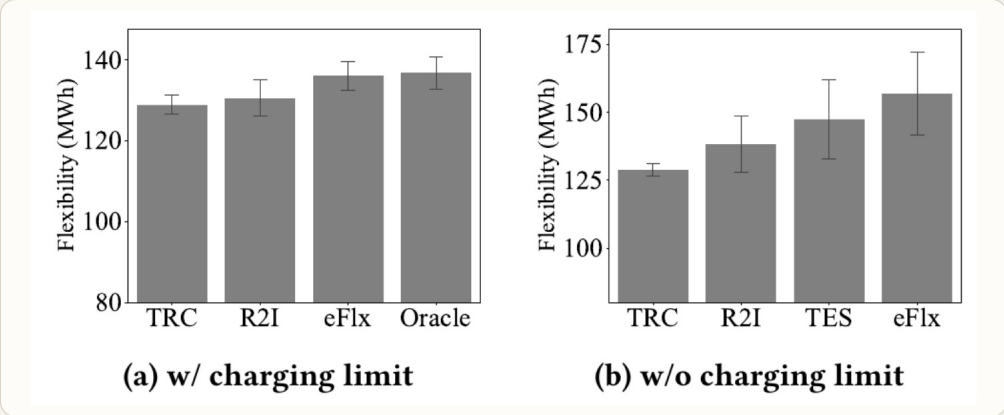
15 battery levels · −1 level per slot driving · +3 per slot charging · DR 17:20–19:20

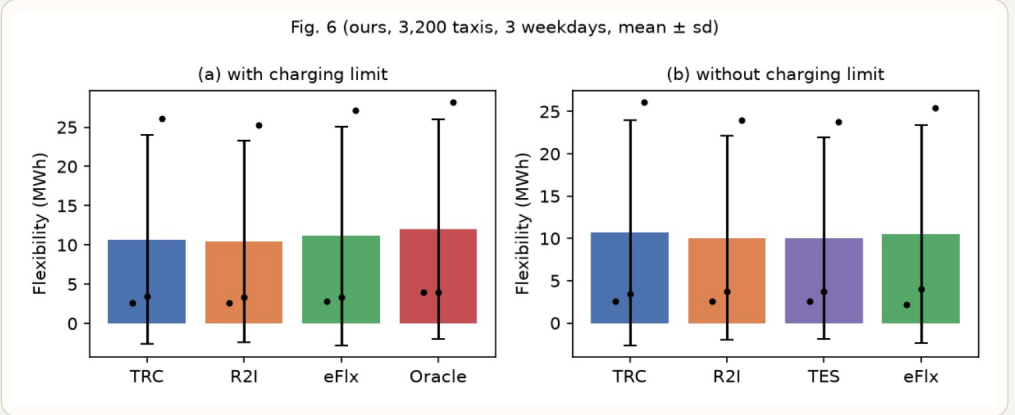
OUR CHOICES

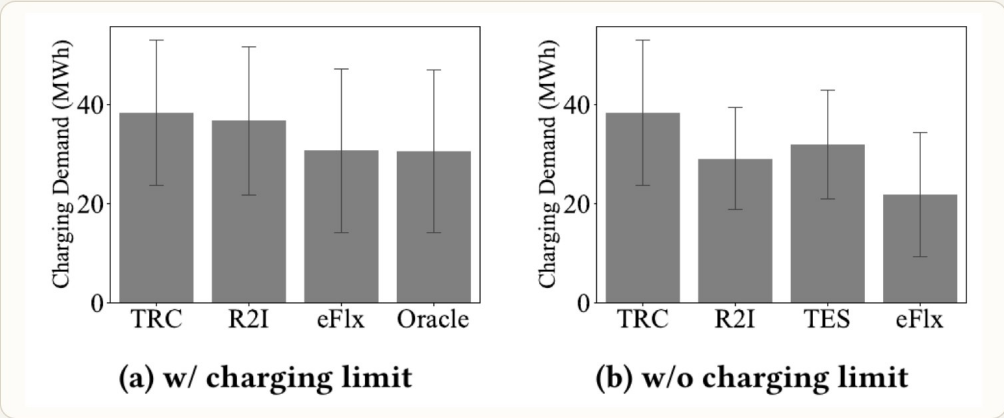
4,000 ports · 3 kWh per level · 12 regions instead of the paper's 38

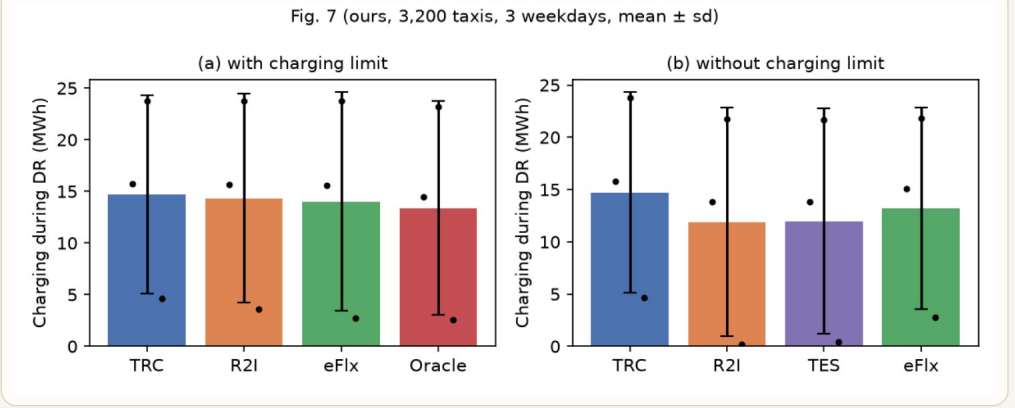
6 · REPRODUCTION

Figs. 6 and 7: paper vs ours (3,200 taxis)









Evaluating eFlx · 18 · left: paper · right: ours, mean ± sd over 3 weekdays

6 · REPRODUCTION

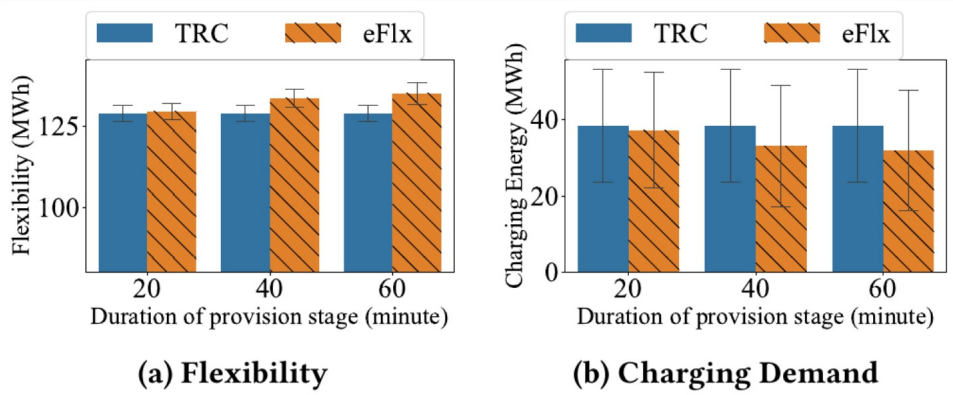
Claim by claim: direction holds, size doesn’t

Claim (eFlx vs ...)	Paper	Ours	Verdict
Flexibility vs TRC	+5.52%	+3.95%	Partly holds
DR charging vs TRC	−19.98%	−4.81%	Direction only
DR charging vs R2I	−16.36%	−2.27%	Direction only
DR charging, gap to Oracle	+0.51%	+4.61%	Does not hold
DR charging vs R2I, no limit	−24.73%	+11.05%	Reversed
DR charging vs TES, no limit	−31.37%	+10.37%	Reversed
Idle driving vs TRC	+2.06%	+9.66%	Higher cost
Waiting at stations vs TRC	+2.12%	+44.14%	Higher cost

Manhattan, 3,200 taxis, totals over 3 weekdays, with the charging limit unless noted. Gains come almost entirely from one day: −1.4%, 0%, −41% vs TRC.

6 · REPRODUCTION

Fig. 11 and Table 1: lead time and forecast error



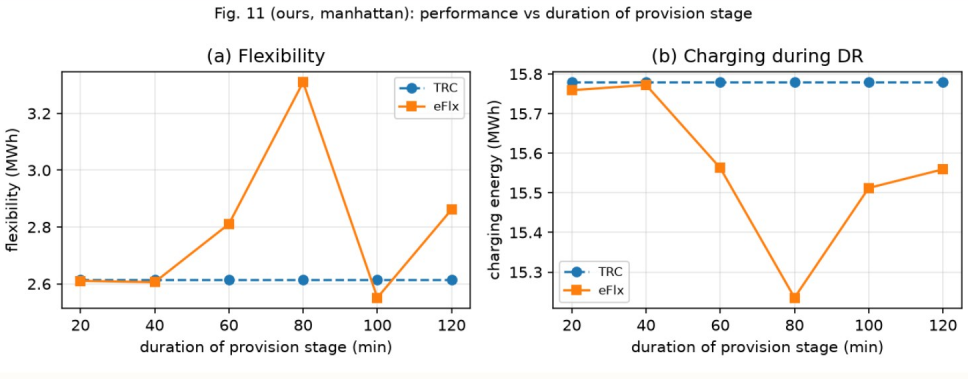


Fig. 11: partly holds

Toy: steady rise, 884 → 1267 kWh. Manhattan: no gain at 20–40 min, peak at 80 min (3.31 vs 2.61 MWh), not monotonic.

Table 1: 10% forecast error

eFlx change	Flexibility	DR charging	Served
Paper	−1.33%	+7.69%	not given
Ours, toy	+3.6%	−7.0%	95.4%
Ours, Manhattan	−6.8%	−3.4%	93.9%

~5% of passengers are missed, and on Manhattan eFlx's edge over TRC almost vanishes (2.67 vs 2.63 MWh).

Evaluating eFlx · 20 · left: paper (top), ours on Manhattan (bottom) · paper_extras.py

7 · NEW USE CASES

New use cases: when does eFlx help?

Fleet-size sweep (Manhattan, DR 17:20–19:20)

Fleet	TRC DR charging	Best possible
13,000 (paper)	—	0 MWh
8,000	100.6 MWh	0 MWh
6,000	66.2 MWh	0 MWh
5,000	59.0 MWh	0 MWh
4,000	35.0 MWh	0 MWh
3,200 (ours)	12.5 MWh	6.6 MWh

The paper’s fleet sits in the “easy” corner of its own Fig. 3

DR at other times

Start at 16:40, 17:20 or 18:00

Several weekdays

Error bars instead of one scenario

Toy city + rebound

Checkable by hand; peak after DR

7 · NEW USE CASE

Absorbing surplus solar and wind power

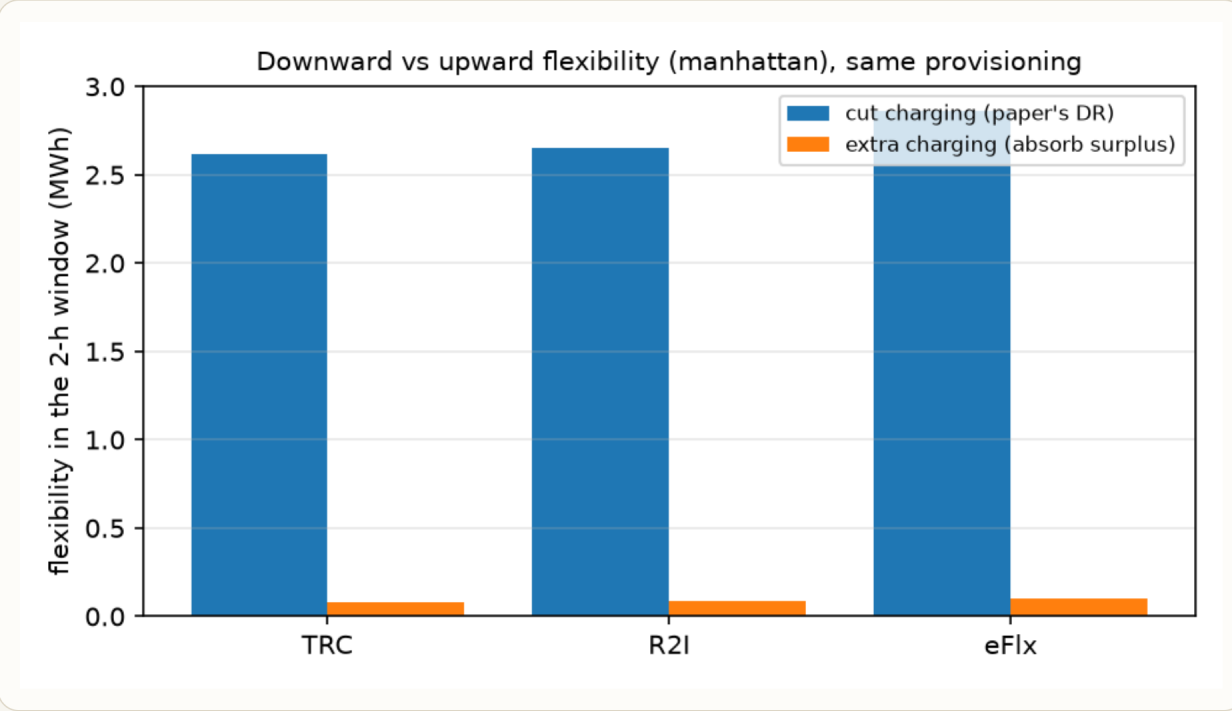
Sometimes the grid has **too much** power (sunny noon, windy night) and wants fleets to charge more. We reverse the response: **maximise** charging in the window, same service rules.

0.1 MWh

extra charging possible, vs 2.6–2.9 MWh of cut (Manhattan)

Why: normal operation already charges at **every free port**; on the toy city there is no room at all.

A fleet selling both directions must deliberately hold charging back: a two-sided plan the paper doesn't consider.



Evaluating eFlx · 22 · Manhattan, 3,200 taxis, weekday 0 · new_ideas.py, results/upward_*.png

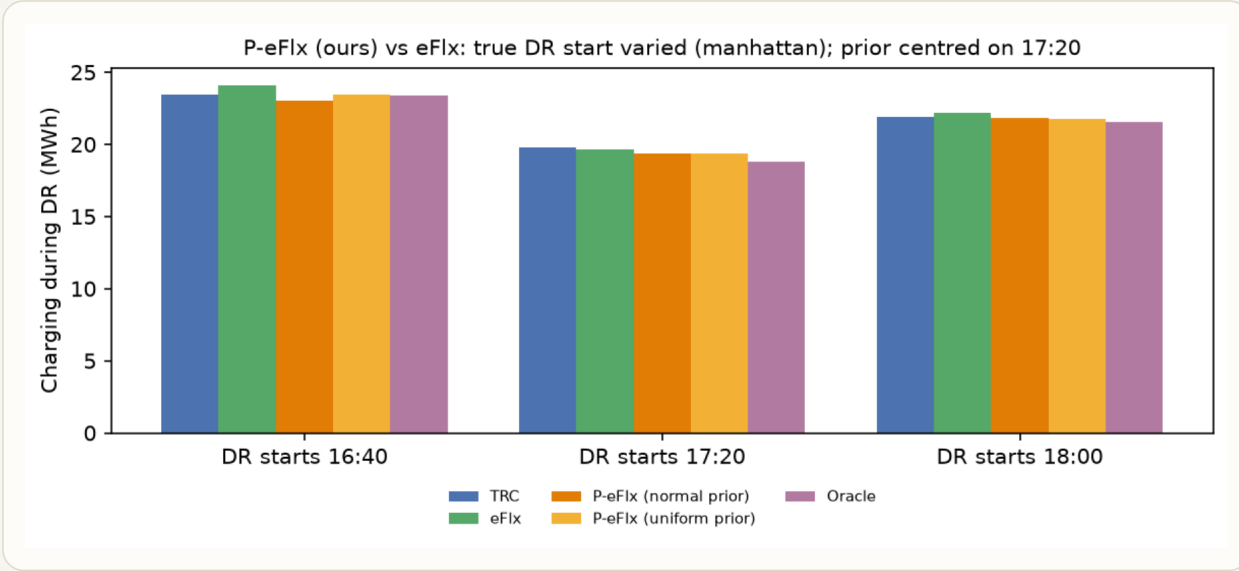
7 · NEW IDEA 1

P-eFlx: plan for when DR is likely

eFlx assumes “DR next slot”. P-eFlx uses a **prior over start times** and minimises **expected** DR charging, at the same LP size.

−32%	DR charging vs eFlx, toy model
−2.6%	DR charging vs eFlx, Manhattan
~75%	of the eFlx–Oracle gap closed, without knowing the start

Caveat: one Manhattan day shows no difference; part of another day’s gain comes with ~1 pp lower service.



7 · NEW IDEA 2

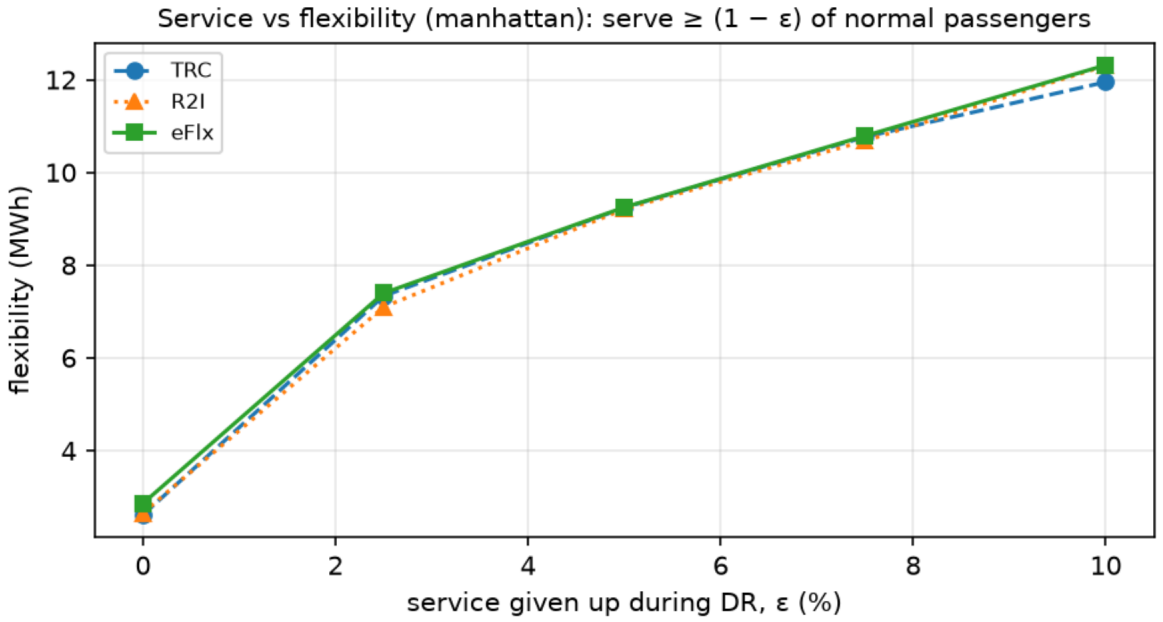
How much flexibility does a little service buy?

The paper mentions a “controlled service compromise” but never measures it. We let the fleet serve only **(1 – ε)** of normal passengers during DR, ε = 0–10%.

2.6× flexibility when 4% of passengers are missed (eFlx 2.86 → 7.40 MWh, Manhattan)

< 1% of service: what eFlx’s provisioning is worth on Manhattan (toy city: ≈ 9%)

For a tight fleet, a small service compromise is a far bigger lever than smarter provisioning, and this curve prices it.



Evaluating eFlx · 24 · Manhattan, 3,200 taxis, weekday 0 · new_ideas.py, results/tradeoff_*.png

8 · CONCLUSION

What the paper did, and what we did

THE PAPER

- Defined the energy flexibility of an e-taxi fleet
- Bi-level provisioning problem, reduced to one LP
- Claimed –19.98% DR charging, 0.52% from Oracle

WE

- Re-implemented it from scratch, on real NYC data
- Reproduced every figure: idea holds, gains shrink (–4.8% vs –19.98%)
- Found gaps; added P-eFlx, service trade-off, surplus absorption

Limits: our parameter choices · 12 regions, not 38 · fluid simulator · few weekdays. **Next:** per-taxi simulator, LP tie-breaking, two-sided DR plans.

TEAM

Contribution

Member	Share	Work
Ghada Suleiman 261330004	33.3%	Paper study: DR, problem, model, eFlx; extension ideas; slides 1–9
Suraj Singh Raman 261110067	33.3%	Claims and critique, implementation, validation, toy experiments, demo, NYC data setup; slides 10–17
Rishikesh Sawant 261110052	33.3%	Manhattan reproduction, new use cases and ideas; slides 18–24

Thank you

Code, data and results: github.com/ascyrax/cs637-eflx